Bottom-Up Analyse – LR(1) Grammatiken

Idee: Ausgehend vom Wort w auf das Startsymbol S reduzieren.

Beispiel: **S → ABC , A→aa | b, B→bbB | ε, C → c**

w = aabbc -> Abbc -> AbbBc -> ABc -> ABC -> S

Umgekehrt ist dies eine **Rechtsableitung:**
S ⇒ABC ⇒ ABc ⇒ AbbBc ⇒ Abbc ⇒ aabbc

LR(k): Lesen des Eingabewortes von links nach rechts mit Lookahead k und Ableiten von rechts nach links.

Vorteil: Mit LR(1)-Grammatiken lassen sich alle deterministisch kontextfreien Sprachen darstellen.

Beispiel im Detail

(1) S → ABC
(2) A → aa
(3) A → b
(4) B → bbB
(5) B → ε
(6) C → c

w = aabbc

	aabbc	schiebe (‘shift’) a
a	abbc	schiebe (‘shift’) a
aa	bbc	reduziere (‘reduce’) aa->A
A	bbc	schiebe b
Ab	bc	schiebe b
Abb	c	schiebe ε
Abb**ε**	c	reduziere ε->B
A**bbB**	c	reduziere bbB->B
AB	c	schiebe c
AB**c**	$	reduziere c->C
ABC	$	reduziere ABC->S
S	$	akzeptiere

Handles
(rechte Seite
einer Ableitungsregel)

Erste Version shift-Reduce Algorithmus

```
while not EOF do
        while not nächstes handle oben auf Stack do
                shift
        end while
        reduce
end while
```

Konflikte

reduce/reduce Konflikt: Es sind gleichzeitig zwei verschiedene Reduce-Schritte möglich.

shift/reduce Konflikt: Ein reduce wäre möglich, aber der Algorithmus kann auch ein shift durchführen, um danach ein anderes handle zu erhalten.

Beispiel

$Stmt \rightarrow if\ E\ Stmt \mid if\ E\ Stmt\ else\ Stmt \mid Assign$

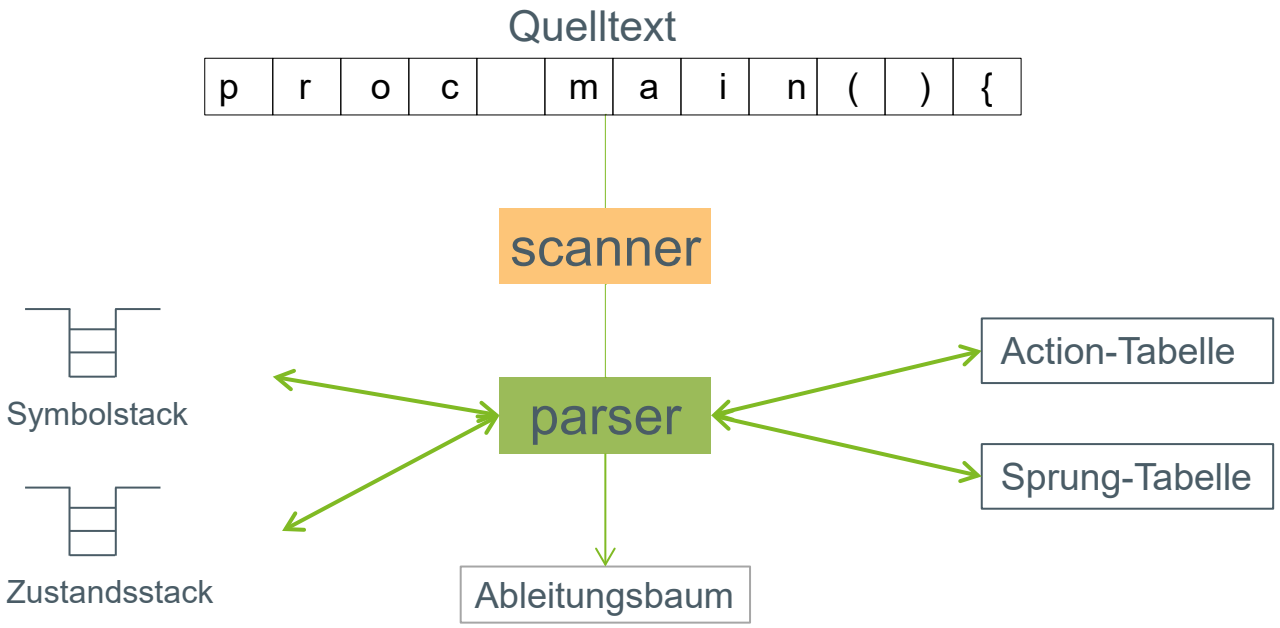
```
if x>0 if y>0 a=1 else a=2
```

Wo tritt ein Konflikt auf?

```
if x>0 if y>0 a=1 else a=2
```

Hier könnte `if y>0 a=1` zu $Stmt$ reduced werden.
Oder: `else` könnte geshifted werden!

LR-Parser schematisch



LR-Syntaxanalyse Algorithmus

```
push(S0);      // Anfangszustand
accept := false;
while (!accept) {
        sei S Zustand oben auf dem Stack;
        sei a das nächste Token;
        switch ( action[s][a] ) {
                case shift(n): push(a,n);              // a auf Symbolstack, n auf Zustandsstack
                        scanner();
                        break;
                case reduce(r): sei A-> β die Regel r;
                        sei k die Länge von β;
                        pop (k Elemente);              // k Symbole, k Zustände vom Stack herunternehmen
                        sei t der Zustand oben auf dem Stack;
                        push(A, Sprung[t][A]);
                        break;
                case accept: accept := true;
                        break;
                case error: error();
                        break;
        }
}
```

Beispiel LR(1) Analysetabelle für E/T/F Grammatik

Zustand	Aktion						Sprung		
	x	+	*	(	)	$	E	T	F
I0	S5			S4			1	2	3
I1		S6				accept			
I2		R2	S7		R2	R2			
I3		R4	R4		R2	R4			
I4	S5			S4			8	2	3
I5		R6	R6		R6	R6			
I6	S5			S4				9	3
I7	S5			S4					10
I8		S6			S11				
I9		R1	S7		R1	R1			
I10		R3	R3		R3	R3			
I11		R5	R5		R5	R5			

S_i : shift und push(Zustand i), r_j: reduce nach Regel j, accept: Akzeptiere, (leer): Error

Konstruktion von LR(0) Parsertabellen

Kernfrage für die Konstruktion der Tabellen: Wie findet man die Zustände?

Definition: Ein **LR(0)-Element** einer Grammatik G ist eine Produktion von G mit einem Punkt an irgendeiner Stelle der rechten Seite.

Beispiel: Zu der Produktion $S \rightarrow ABC$ gibt es vier Elemente:

$S \rightarrow .ABC$, $S \rightarrow A.BC$, $S \rightarrow AB.C$, $S \rightarrow ABC.$

Idee: Die Symbole links des Punktes sind schon auf dem Symbolstack!

Definition: Eine Menge von Elementen heißt **Kollektion**.

Kollektionen dienen als Zustände des Automaten.

Hüllenkonstruktion

Definition: Sei I eine Menge von LR(0)-Elementen. Hülle(I) für eine Grammatik G ist die Menge von Elementen, die aus I durch eine der folgenden Regeln konstruiert wird:

(I) Jedes Element aus I gehört zu Hülle(I)

(II) Wenn $A \to \alpha.\mathrm{B}\beta \in Hülle(I)\ und\ B \to \gamma\ eine\ Produktion, dann\ ist$ $B \to .\gamma \in Hülle(I)$.

Wenn in I ein Element mit einem Nichtterminal direkt rechts eines Punktes ist, enthält die Hülle auch Elemente mit den Regeln für dieses Nichtterminal beginnend mit einem Punkt auf der linken Seite der Regel.

Wenn $A \to \alpha.\mathrm{B}\beta \in Hülle(I)$, dann erwartet der Parser, dass er ein Teilwort lesen wird, das von $\mathrm{B}\beta$ ableitbar ist. Wenn $B \to \gamma$ eine Produktion ist, dann bedeutet dies, dass der Parser einen Substring lesen wird, der von γ ableitbar ist.

Sprung-Funktion

Definition: Sei I eine Menge von Elementen und X ein Terminal- oder nicht-Terminalsymbol, dann ist **Sprung(I, X)** die Hülle der Menge aller Elemente $A \rightarrow \alpha X . \beta$, so dass $A \rightarrow \alpha . X\beta$ in I ist.

D.h., ein Sprung(I, X) ist nur möglich, wenn I Elemente enthält, die einen Punkt direkt vor X haben.

Definition: Ein **Kernelement** ist entweder die Startproduktion S' → .S oder ein Element, bei denen der Punkt auf der rechten Regelseite nicht am Anfang steht.

Berechnung der Sprung-Funktion

Berechnung von Sprung(I, X):

J := ∅
Wiederhole für jedes Element A → u.Xw in I
J := J U { A → uX.w }
Sprung(I, X)= Hülle(J)

LR(0)-Tabellenkonstruktion

Algorithmus:

C:={ Hülle({S' →.S$})}
Wiederhole
  Für jeden Zustand I **in** C
    Für jedes Element $A \rightarrow \alpha.X\beta$ **in** I
      Wenn Sprung(I, X) ≠ ∅ **dann**
        füge Sprung(I,X) zu C hinzu
Solange, **bis** sich nichts mehr an C ändert.

C heißt **kanonische Kollektion** von G.

Beispiel (unvollständig)

Grammatik : E‘→ E,
E → E+T | T,
T → T*F | F,
F → (E) | x

Dann ergibt sich nach Anwendung des Algorithmus’ :
$C = \{ I_o, \dots, I_{11} \} mit$

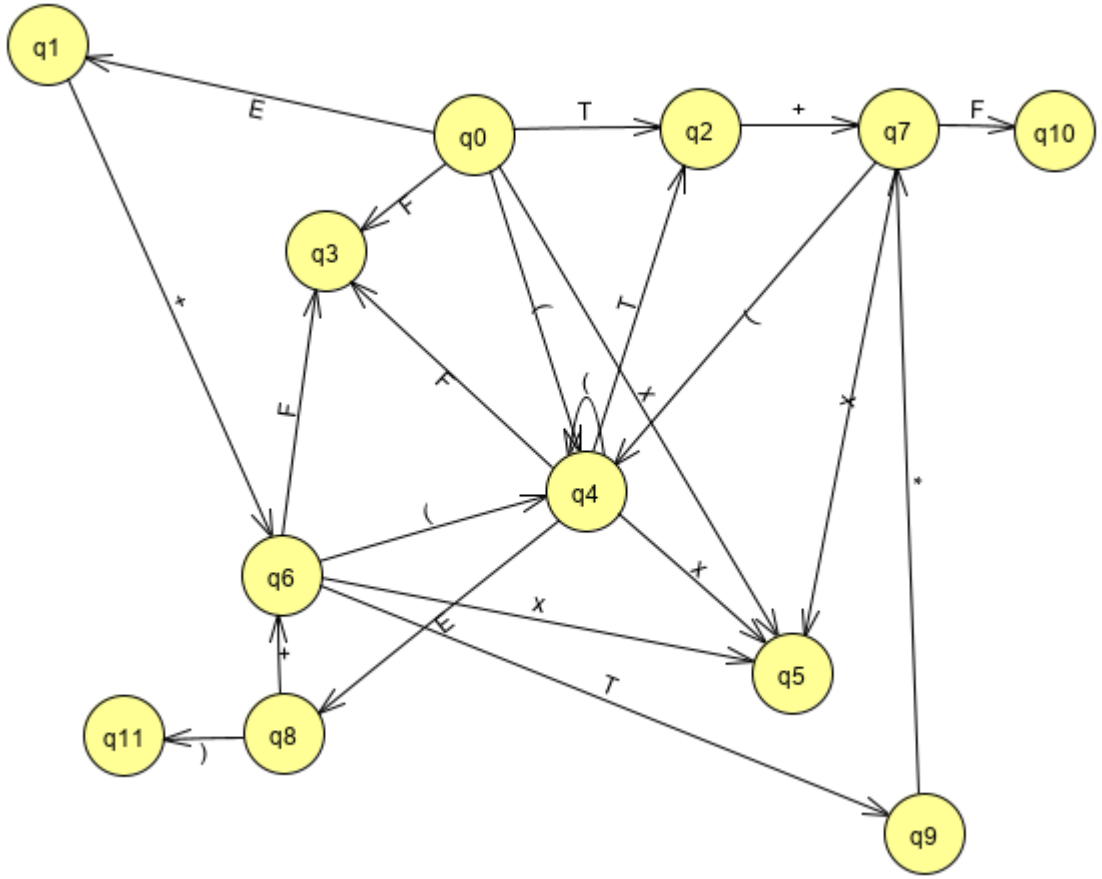
I_o = Hülle({E’ →. E$)} = { E’ → .E, E → .E+T, E → .T, T → .T∗F, T →. F, F → . (E), F → . x }
I_1 = Sprung(I_o, E) = Hülle({E’ → E., E → E.+T})= { E' → E. , E → E. +T}
I_2 = Sprung(I_o, T) = Hülle({E → T., T → T.∗F}) = {E → T., T → T.∗ F}
…
$I_5 = Sprung(I_0, x) = Hülle(\{F \to x.\}) = \{F \to x.\}$
…
$I_{11} = Sprung(I_8, ')')$= …

Beispiel: Unvollständige LR(0)-Analysetabelle

Zustand	Action					Sprung		
	x	+	*	(	)	E	T	F
I0	S I5			S I4		I1	I2	I3
I1		S I6						
I2			S I7					
I3								
I4	S I5			S I4		I8	I2	I3
I5								
I6	S I5			S I4			I9	I3
I7	S I5			S I4				I10
I8		S I6			S I11			
I9			S I7					
I10								
I11								

S I_j : shift und push(Zustand j), r_j: reduce nach Regel j, accept: Akzeptiere, (leer): Error

Zustandsdiagramm



Berechnung der Reduktionen

Algorithmus:

R := ∅;

Für jeden Zustand I in C

Für (jedes Element A → w. in I)

R := R U { (I, A → w) };

Im Beispiel oben treten Elemente A → w. nur in I1, I2, I3, I5, I9, I10, I11 auf. Die Reduktionen sind daher:

R = { (I1, E' → E), (I2, E → T), (I3, T → F),
(I5, F → x), (I9, E → E+T), (I10, T → T*F), (I11, F → E) }

Beispiel LR(0)-Analysetabelle mit Reduce Actions

Zustand	Action						Sprung		
	x	+	*	(	)	Reduce	E	T	F
I0	S I5			S I4			I1	I2	I3
I1		S I6				Reduce E'->E			
I2			S I7			Reduce E->T			
I3						Reduce T->F			
I4	S I5			S I4			I8	I2	I3
I5						Reduce F->x			
I6	S I5			S I4				I9	I3
I7	S I5			S I4					I10
I8		S I6			S I11				
I9			S I7			Reduce E->E+T			
I10						Reduce T->T*F			
I11						ReduceF->(E)			

S I_j : shift und push(Zustand j), r_j: reduce nach Regel j, accept: Akzeptiere, (leer): Error

SLR (Simple LR)

Wie man am vorigen Beispiel sieht, ist eine **Vorausschau** notwendig, um die Shift/Reduce-Konflikte zu umgehen.

Ein SLR(1) Parser verwendet 1 Token zur Vorausschau.

Unterschied zum LR(0)-Parser:

Reduktion bzgl. Regel A → w wird nur für die Symbole in der Folgemenge von A eingetragen.

Im Beispiel oben: Follow(E') = { $ }
Follow(E) = { +,), $ }
Follow(T) = { +, *,), $ }
Follow(F) = {+, *,), $ }

Beispiel SLR(1)-Analysetabelle mit Reduce Actions

Zustand	Action						Sprung		
	x	+	*	(	)	$	E	T	F
I0	S I5			S I4			I1	I2	I3
I1		S I6				Accept (Reduce E'->E)			
I2		R2	S I7		R2	R2 (E->T)			
I3		R 4	R 4		R4	R4 (T->F)			
I4	S I5			S I4			I8	I2	I3
I5		R6	R6		R6	R6 (F->x)			
I6	S I5			S I4				I9	I3
I7	S I5			S I4					I10
I8		S I6			S I11				
I9		R1	S I7		R1	R1 (E->E+T)			
I10		R3	R3		R3	R3 (T->T*F)			
I11		R5	R5		R5	R5 (F->(E))			

S I_j : shift und push(Zustand j), r_j: reduce nach Regel j, accept: Akzeptiere, (leer): Error

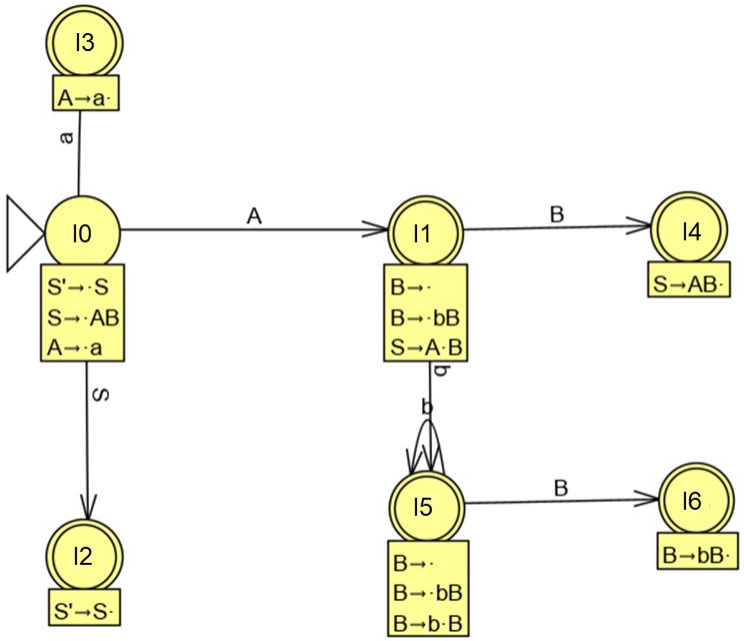


Aufgabe: Berechnen Sie die kanonische Kollektion

S‘ → S, S → AB, A → a, B → bB, B → ε

I0 = Hülle({S‘ -> .S$}) = { S‘ -> .S$, S -> .AB, A -> .a }
I1 = Sprung(I0, A) = Hülle({S -> A.B}) = { S -> A.B, B -> .bB, B -> . }
I2 = Sprung(I0, S) = Hülle({S‘ -> S.}) = { S‘ -> S. }
I3 = Sprung(I0, a) = Hülle({A -> a.}) = { A -> a. }
I4 = Sprung(I1, B) = Hülle({S -> AB.}) = { S -> AB. }
I5 = Sprung(I1, b) = Hülle ({B -> b.B}) = { B -> b.B, B -> .bB, B->. }
I6 = Sprung(I5, B) = Hülle ({B->bB. }) = { B -> bB. }
Sprung(I5, b) = Hülle ({B->b.B}) = I5

Ergebnis: Automat



Ergebnis: Berechnen der Reduce-Aktionen und Tabelle

I0 = { S‘ -> .S$, S -> .AB, A -> .a }
I1 = { S -> A.B, B -> .bB, B -> . }
I2 = { S‘ -> S. }
I3 = { A -> a. }
I4 = { S -> AB. }
I5 = { B -> b.B, B -> .bB, B->. }
I6 = { B -> bB. }
=>
R = { (I1, B -> ε), (I2, S‘->S), (I3, A->a),
(I4, S->AB), (I5, B -> ε),
(I6, B->bB) }

	Action			Sprung		
	a	**b**	**$**	**A**	**B**	**S**
I0	s3			I1		I2
I1		s5	reduce B->ε		I4	
I2			accept			
I3		reduce A->a	reduce A->a			
I4			reduce S->AB			
I5		s5	Reduce B->ε		I6	
I6			Reduce B->bB			

Follow(A)={b, $}, Follow(B)=Follow(S)=Follow(S‘)={$}

Anwendung des Shift-Reduce-Algorithmus

Symbolstack	Zustandsstack	Restliches Eingabewort	Aktion
	0	abb	s3
a	0 3	bb	reduce A->a, Sprung 1
A	0 1	bb	s5
Ab	0 1 5	b	s5
Abb	0 1 5 5	$	reduce B->ε, Sprung 6
AbbB	0 1 5 5 6	$	reduce B->bB, Sprung 6
AbB	0 1 5 6	$	reduce B->bB, Sprung 4
AB	0 1 4	$	reduce S->AB, Sprung 2
S	0 2	$	accept

SLR = KFG ?

Problem der SLR-Grammatiken: Es gibt eindeutige Grammatiken, die nicht SLR sind, die also zu Konflikten führen.

Die Konstruktionsmethode für die SLR-Tabelle ist nicht mächtig genug, um dem Parser genügend Kontext mitzugeben, damit er entscheiden kann, welche der Aktionen ausgeführt werden soll.

Beispiel:

S -> aXa | bXb

X -> a | ε

Im Zustand q2 führt das Element X->.a dazu, dass ein shift a möglich ist, aber wegen X->. ist auch ein reduce möglich (dito bei q3)! Da a zur Folgemenge von X gehört, müssen beide in die gleiche Zelle eintragen werden!

